

BAHAN AJAR - PERTEMUAN 12

Graph, BFS & DFS

TP	BK, AP — Strategi Algoritmik
----	------------------------------

A. APA ITU GRAPH?

Graph adalah struktur data yang terdiri dari **verteks** (simpul/node) dan **edge** (sisi) yang menghubungkan verteks-verteks tersebut.

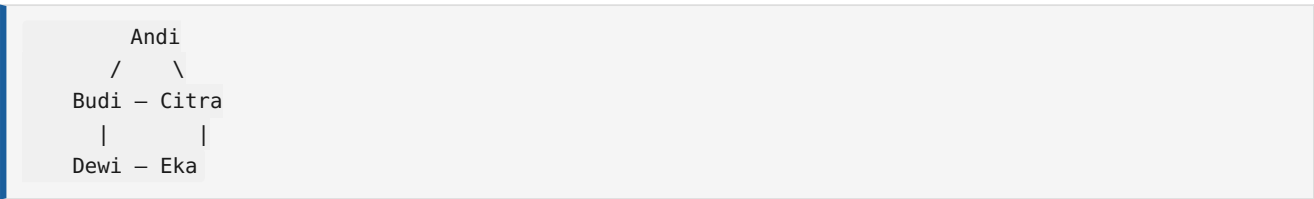
Notasi: **G = (V, E)**

- $V = \{v_1, v_2, \dots, v_n\}$ = himpunan verteks
- $E = \{e_1, e_2, \dots, e_m\}$ = himpunan edge

Istilah Penting

Istilah	Arti	Contoh
Vertex (verteks)	Simpul/node	Orang, kota, halaman web
Edge	Sisi/hubungan	Pertemanan, jalan, hyperlink
Derajat (degree)	Jumlah edge terhubung ke verteks	Jumlah teman seseorang
Path	Jalur dari verteks A ke B	Rute perjalanan
Cycle	Jalur yang kembali ke verteks awal	Lingkaran pertemanan
Connected	Ada path antara setiap verteks	Semua terhubung

Analogi: Media Sosial



Graph	Media Sosial
Verteks	Akun (Andi, Budi, Citra, Dewi, Eka)
Edge	Pertemanan
Derajat Andi	2 (Budi & Citra)
Derajat Budi	3 (Andi, Citra, Dewi)

B. JENIS-JENIS GRAPH

1. Graph Tak Berarah (Undirected Graph)

Edge tidak memiliki arah — hubungan dua arah.

```
A — B — C
|       |
D — E — F
```

Contoh: Facebook pertemanan — jika A teman B, maka B juga teman A.

2. Graph Berarah (Directed Graph / Digraph)

Edge memiliki arah (panah). Hubungan satu arah.

```
A → B → C
↑       ↓
D ← E ← F
```

Contoh: Twitter follow — A follow B, belum tentu B follow A.

3. Graph Berbobot (Weighted Graph)

Setiap edge memiliki bobot/nilai.

```
A —5— B —3— C
|       |
2       4
|       |
D —1— E —6— F
```

Contoh: Google Maps — bobot = jarak/waktu tempuh.

4. Rangkuman Jenis

Jenis	Arah?	Bobot?	Contoh
Tak berarah	× Dua arah	×	Facebook, WhatsApp group
Berarah	Satu arah	×	Twitter, Instagram follow
Berbobot	×	atau	Google Maps, jaringan listrik
Lengkap	Semua terhubung	×	Turnamen round-robin

C. REPRESENTASI GRAPH

1. Adjacency Matrix

Matriks $V \times V$. Baris = asal, kolom = tujuan. 1 = ada edge, 0 = tidak ada.

	A	B	C	D
A	[0 1 1 0]			
B	[1 0 1 0]			
C	[1 1 0 1]			
D	[0 0 1 0]			

Untuk graph berbobot: isi dengan bobot (∞ jika tidak terhubung).

Kelebihan	Kekurangan
Cek edge $O(1)$	Memori $O(V^2)$ — boros untuk graph jarang
Sederhana	$V=1000 \rightarrow$ matriks 1 juta sel

2. Adjacency List

Array/daftar yang berisi tetangga setiap verteks.

```
A: [B, C]
B: [A, C]
C: [A, B, D]
D: [C]
```

Kelebihan	Kekurangan
Hemat memori $O(V+E)$	Cek edge $O(V)$
Cocok untuk graph jarang	Sedikit lebih kompleks

D. BFS — BREADTH-FIRST SEARCH

BFS menjelajah graph **per level** — semua tetangga terdekat dikunjungi dulu, baru tetangga yang lebih jauh.

Algoritma

```
PROCEDURE BFS(graph, start)
  visited ← []           // tandai sudah dikunjungi
  queue ← [start]        // antrean (FIFO)

  WHILE queue tidak kosong DO
    v ← queue.hapusDepan()
    IF v belum dikunjungi THEN
      tandai v sebagai dikunjungi
      FOR each tetangga u dari v DO
        IF u belum dikunjungi THEN
          queue.tambah(u)
        ENDIF
      ENDFOR
    ENDIF
  ENDWHILE
END
```

Contoh BFS — Graph Sosial

```

A - B - C
|       |
D - E - F

```

BFS dari A:

Langkah	Queue	Kunjungan
0	[A]	—
1	[B, D]	A
2	[D, C]	A, B
3	[C, E]	A, B, D
4	[E, F]	A, B, D, C
5	[F]	A, B, D, C, E
6	[]	A, B, D, C, E, F

Urutan BFS dari A: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

Karakteristik BFS

- **Queue (FIFO)** — first in, first out
- Menemukan **jalur terpendek** (dalam edge) dari start
- Kompleksitas: $O(V+E)$
- Cocok untuk: mencari teman terdekat, GPS, web crawler

E. DFS — DEPTH-FIRST SEARCH

DFS menjelajah graph dengan cara **masuk ke dalam (depth)** dulu sebelum mundur.

Algoritma (Stack)

```

PROCEDURE DFS(graph, start)
  visited ← []
  stack ← [start]      // tumpukan (LIFO)

  WHILE stack tidak kosong DO
    v ← stack.hapusAtas()
    IF v belum dikunjungi THEN
      tandai v sebagai dikunjungi
      FOR each tetangga u dari v DO
        IF u belum dikunjungi THEN
          stack.tambah(u)
        ENDIF
      ENDFOR
    ENDIF
  ENDWHILE
END

```

Algoritma (Rekursif)

```

PROCEDURE DFS_Rekursif(graph, v)
    tandai v sebagai dikunjungi

    FOR each tetangga u dari v DO
        IF u belum dikunjungi THEN
            DFS_Rekursif(graph, u)
        ENDIF
    ENDFOR
END

```

Contoh DFS — Graph Sosial

```

A — B — C
|         |
D — E — F

```

DFS dari A (gunakan stack, tetanggaurut abjad):

Langkah	Stack	Kunjungan
0	[A]	—
1	[B, D]	A
2	[B, E]	A, D (ambil D dari stack)
3	[B]	A, D, E
4	[]	A, D, E, B
	(B tetangganya A, C, E — A & E sudah visited, C belum)	
4	[C]	A, D, E, B
5	[F]	A, D, E, B, C
6	[]	A, D, E, B, C, F

Urutan DFS dari A: A → D → E → B → C → F

F. BFS vs DFS

Aspek	BFS	DFS
Struktur data	Queue (FIFO)	Stack (LIFO) / Rekursi
Cara jelajah	Level by level	Masuk terdalam dulu
Jalur terpendek?	Ya (dalam edge)	
Memori	O(V) — simpan 1 level	O(V) — simpan 1 path
Cocok untuk	Graph rapat, jarak pendek	Graph jarang, eksplorasi
Contoh	Google Maps, Facebook friends	Puzzle, maze, AI game

G. RANGKUMAN

Konsep	Inti
Graph	V (verteks) + E (edge)
Tak berarah	Hubungan dua arah
Berarah	Hubungan satu arah
Berbobot	Edge punya nilai
Adjacency Matrix	Cek edge $O(1)$, memori $O(V^2)$
Adjacency List	Hemat memori $O(V+E)$
BFS	Queue — level by level — jalur terpendek
DFS	Stack — depth first — eksplorasi

MGMP Informatika SMAN 6 Cimahi